



CSC1029 OOProgramming

Lecture 7 – Object Design & Implementation



Before we begin



The story so far ...

Objects classes - 'blueprints' for objects

Instance data - to define object state

Instance methods - to define behaviour

Access modifiers - public/private

Encapsulation - private data/get/set

Constructor - methods

Instantiation - creation of objects

UML - charts to describe classes

Before we begin – any questions?



... and a *reminder*:

The first assessment
(practical test) will take
place *during the lab session*
on Wednesday 15th Feb 2023.

You will be asked to implement
a solution to a problem –
specified using UML.



What's to come in today's lecture?

- Development of a Student Class
- Development of a Menu Class
- Robust programs & The need for Exception handling

Introducing a Class to Describe Students⁽¹⁾

Student
- studentId : int - name : String - course : String - faculty : char
+ Student(int, String, String, char) + getStudentId() : int + getName() : String + getCourse() : String + getFaculty() : char + setCourse(String) : void + setFaculty(char) : void

```
public class Student {  
    private int studentId;  
    private String name;  
    private String course;  
    private char faculty;  
  
    public Student(int id, String sname, String  
                    scourse, char fac) {  
        studentId = id;  
        name = sname;  
        course = scourse;  
        faculty = fac;  
    }  
  
    ...  
}
```

Introducing a Class to Describe Students ⁽²⁾

Student

- studentId : int
- name : String
- course : String
- faculty : char

- + Student(int, String, String, char)
- + getStudentId() : int
- + getName() : String
- + getCourse() : String
- + getFaculty() : char
- + setCourse(String) : void
- + setFaculty(char) : void

```
public class Student {  
    private int studentId;  
    private String name;  
    private String course;  
    private char faculty;
```

```
    public Student(int id, String sname, String  
                    scourse, char fac) {  
        this.studentId = id;  
        this.name = sname;  
        this.course = scourse;  
        this.faculty = fac;  
    }
```

```
    // use the 'this' reference to refer to the  
    // object under construction
```

```
}
```

Introducing a Class to Describe Students (3)

Student

```
- studentId : int  
- name : String  
- course : String  
- faculty : char
```

```
+ Student(int, String,  
String, char)  
+ getStudentId() : int  
+ getName() : String  
+ getCourse() : String  
+ getFaculty() : char  
+ setCourse(String) : void  
+ setFaculty(char) : void
```

// Getters

```
public int getStudentId() {  
    return studentId;  
}  
public String getName() {  
    return name;  
}  
public String getCourse() {  
    return course;  
}  
public char getFaculty() {  
    return faculty;  
}
```

Introducing a Class to Describe Students (4)

Student
- studentId : int - name : String - course : String - faculty : char
+ Student(int, String, String, char) + getId() : int + getName() : String + getCourse() : String + getFaculty() : char + setCourse(String) : void + setFaculty(char) : void

```
// Setters
public void setName(String aName) {
    name = aName;
}

public void setCourse(String aCourse) {
    course = aCourse;
}

public void setFaculty(char aFaculty) {
    faculty = aFaculty;
}
```

Do we need to include any parameter validation?

Introducing a Class to Describe Students (5)

```
public class StudentTester {  
  
    public static void main(String[] args) {  
        Student s1 = new Student(12345, "Homer Simpson", "Computer Science", 'S');  
        Student s2 = new Student(54321, "Bart Simpson", "Software Engineering", 'S');  
  
        String data = "Number: " + s1.getId() + "\n";  
        data += "Name: " + s1.getName() + "\n";  
        data += "Course: " + s1.getCourse() + "\n";  
        data += "Faculty: " + s1.getFaculty() + "\n";  
  
        System.out.println(data);  
    }  
}
```

Console Output

```
Number: 12345  
Name: Homer Simpson  
Course: Computer Science  
Faculty: S
```

Introducing a Class to Describe Students ⁽⁶⁾

Student

- studentId : int
- name : String
- course : String
- faculty : char

- + Student(int, String, String, char)
- + getStudentId() : int
- + getName() : String
- + getCourse() : String
- + getFaculty() : char
- + setCourse(String) : void
- + setFaculty(char) : void
- + toString() : String

```
public class Student {  
    private int studentId;  
    private String name;  
    private String course;  
    private char faculty;  
  
    public String toString() {  
        String data = "Number: " + getStudentId()  
                      + "\n";  
  
        data += "Name: " + getName() + "\n";  
        data += "Course: " + getCourse() + "\n";  
        data += "Faculty: " + getFaculty() + "\n";  
        return data;  
    }  
  
    ...  
}
```

Introducing a Class to Describe Students (7)

```
public class StudentTester {  
  
    public static void main(String[] args) {  
        Student s1 = new Student(12345, "Homer Simpson", "Computer Science", 'S');  
        Student s2 = new Student(54321, "Bart Simpson", "Software Engineering", 'S');  
  
        String data = "Number: " + s1.getId() + "\n";  
        data += "Name: " + s1.getName() + "\n";  
        data += "Course: " + s1.getCourse() + "\n";  
        data += "Faculty: " + s1.getFaculty() + "\n";  
  
        System.out.println(s2);  
    }  
}
```

Console Output

```
Number: 54321  
Name: Bart Simpson  
Course: Software Engineering  
Faculty: S
```

Exercise – Designing a Menu Class ⁽¹⁾

Suppose we need to represent a simple console-based **Menu** ...

Console Output

Student Registration

+++++

1. Add New Student

2. List All Students

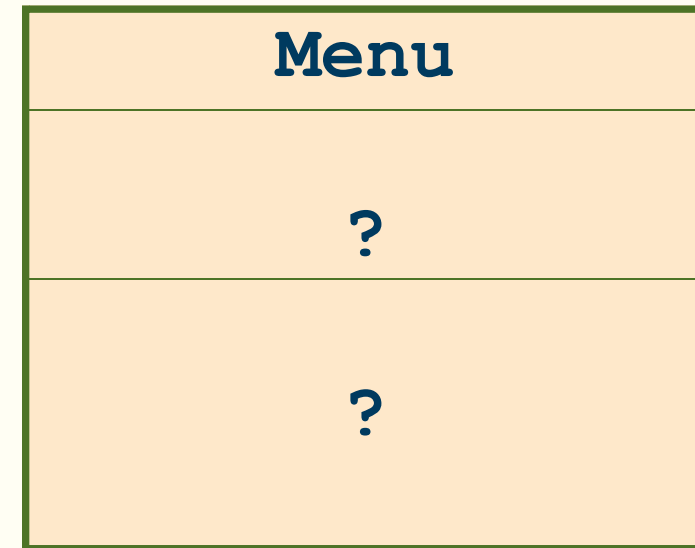
3. Quit

Enter Selection: 2

User picked option: 2

What **instance data** would we need?

What **behaviour** would be appropriate?



Exercise – Designing a Menu Class ⁽²⁾

Suppose we need to represent a simple console-based **Menu** ...

Console Output

Student Registration

+++++

1. Add New Student

2. List All Students

3. Quit

Enter Selection: 2

User picked option: 2

Title

Options

Obtaining a
selection from
the user

What **instance data** would we need?

What **behaviour** would be appropriate?

Menu

- items: String[]
- title : String
- input : Scanner
- display() : void
- + getUserChoice():int

Implementing a Menu Class ⁽¹⁾

Menu

- items: String[]
- title : String
- input : Scanner

- display() : void
- + getUserChoice():int

```
public class Menu {  
    private String items[];  
    private String title;  
    private Scanner input;  
  
    public Menu(String title, String data[]) {  
        this.title = title;  
        this.items = data;  
        this.input = new Scanner(System.in);  
    }  
  
    ...  
  
    // There are some potential problems with  
    // this constructor - to be discussed again  
  
}
```

Implementing a Menu Class ⁽²⁾

Menu

- items: String[]
- title : String
- input : Scanner

- display() : void
- + getUserChoice():int

```
public class Menu {  
    private String items[];  
    private String title;  
    private Scanner input;  
  
    public int getUserChoice() {  
        display();  
        System.out.print("Enter Selection: ");  
        int value = input.nextInt();  
        return value;  
    }  
    ...  
}
```

Implementing a Menu Class ⁽³⁾

Menu

- items: String[]
- title : String
- input : Scanner

- display() : void
- + getUserChoice():int

```
public class Menu {  
    private String items[];  
    private String title;  
    private Scanner input;  
  
    private void display() {  
        System.out.println(title);  
        for(int i=0;i<title.length();i++) {  
            System.out.print("+");  
        }  
        System.out.println();  
        for(int opt=1;opt<=items.length; opt++) {  
            System.out.println(opt + ". " +  
                               items[opt-1] );  
        }  
        System.out.println();  
    }  
    ...  
}
```


Implementing a Menu Class ⁽⁴⁾

```
public class MenuTester {  
  
    public static void main(String[] args) {  
        String title = "Student Registration";  
        String options[] = {"Add New Student",  
                             "List All Students", "Quit"};  
  
        Menu regMenu = new Menu(title, options);  
        int choice = regMenu.getUserChoice();  
  
        System.out.println("User picked option: "  
                           + choice);  
    }  
}
```

Console Output

```
Student Registration  
+++++  
1. Add New Student  
2. List All Students  
3. Quit  
  
Enter Selection: 2  
User picked option: 2
```

Implementing a Menu Class ⁽⁵⁾

```
public class MenuTester {  
  
    public static void main(String[] args) {  
        String title = "Student Registration";  
        String options[] = {"Add New Student",  
                             "List All Students", "Quit"};  
  
        Menu regMenu = new Menu(title, options);  
        int choice = regMenu.getUserChoice();  
  
        System.out.println("User picked option: "  
                           + choice);  
    }  
}
```

Console Output

```
Student Registration  
+++++  
1. Add New Student  
2. List All Students  
3. Quit  
  
Enter Selection: 4  
User picked option: 4
```

What's the problem here?

Implementing a Menu Class ⁽⁶⁾

```
public class MenuTester {  
  
    public static void main(String[] args) {  
        String title = "Student Registration";  
        String options[] = {"Add New Student",  
                            "List All Students", "Quit"};  
  
        Menu regMenu = new Menu(title, options);  
        int choice = regMenu.getUserChoice();  
  
        System.out.println("User picked option: "  
                           + choice);  
    }  
}
```

Console Output

```
Student Registration  
+++++  
1. Add New Student  
2. List All Students  
3. Quit
```

```
Enter Selection: hello
```

```
Exception in thread "main" java.util.InputMismatchException  
    at java.util.Scanner.throwFor(Scanner.java:864)  
    at java.util.Scanner.next(Scanner.java:1485)  
    at java.util.Scanner.nextInt(Scanner.java:2117)  
    at java.util.Scanner.nextInt(Scanner.java:2076)  
    at week3.Menu.getUserChoice(Menu.java:31)  
    at week3.MenuTester.main(MenuTester.java:9)
```

What's the problem here?



What's Next?

Implementing Exception Handling

Arrays of Objects

Object References - Memory Organisation

